

DEEP LEARNING

Concepts, Meanings & Short Answers

Not dependent on practical output — Pure conceptual understanding

Student:

Manas Suhas Ohal

Roll No:

75 | BE-B

Covers:

Assignment 1 (Boston) · Assignment 2 (IMDB) · Assignment 3 (CNN) · Additional

ASSIGNMENT 1 — LINEAR REGRESSION USING DEEP NEURAL NETWORK

Dataset: Boston Housing Price Prediction

[Core DNN Concepts]

1 What is a Deep Neural Network (DNN)?

A DNN is a machine learning model inspired by the human brain. It has multiple layers of neurons — an input layer, one or more hidden layers, and an output layer. Each neuron receives inputs, multiplies them by weights, adds a bias, passes the result through an activation function, and sends the output forward. 'Deep' means it has more than one hidden layer.

2 What is the Input Layer?

The input layer is the first layer of the network. It receives the raw data — one neuron per feature. For Boston Housing, the input layer has 13 neurons (one for each feature like crime rate, rooms, tax, etc.). No computation happens here — it simply passes data to the next layer.

3 What is a Hidden Layer?

Hidden layers are the layers between input and output. They perform mathematical transformations (weighted sum + activation) to learn complex patterns from data. More hidden layers = deeper network = ability to learn more complex relationships. In our model: two hidden layers with 128 and 64 neurons.

4 What is an Output Layer?

The output layer is the final layer that gives the prediction. For regression (Boston Housing), it has 1 neuron with linear activation — outputs any real number (the predicted price). For classification, it would have as many neurons as classes, with softmax or sigmoid activation.

5 What is Sum of Weighted Inputs?

Each neuron computes: $z = (w_1 \times x_1) + (w_2 \times x_2) + \dots + (w_n \times x_n) + b$. This is called the weighted sum or linear combination. w are the weights (learned during training), x are inputs, and b is the bias term. This z is then passed through the activation function to introduce non-linearity.

[Activation Functions]

6 What is ReLU? Why is it used?

ReLU (Rectified Linear Unit) = $\max(0, x)$. If input is positive, output equals input. If negative, output is 0. It is used in hidden layers because: (1) It does not saturate for positive values, so gradients flow freely during backpropagation. (2) Computationally simple — no exponential. (3) Solves the vanishing gradient problem that sigmoid/tanh suffer from.

7 What is Sigmoid activation?

Sigmoid = $1 / (1 + e^{-x})$. Outputs a value between 0 and 1. Used in the output layer for binary classification — the output represents a probability. Problem: saturates near 0 and 1, causing vanishing gradient in hidden layers. That's why ReLU is preferred for hidden layers.

8 What is Softmax activation?

Softmax converts raw output scores (logits) of multiple neurons into probabilities that sum to 1. Used in output layer for multi-class classification. Formula: $\text{softmax}(z_i) = \exp(z_i) / \sum(\exp(z_j))$. The class with the highest probability is the prediction.

9 What is Tanh activation?

Tanh = $(e^x - e^{-x}) / (e^x + e^{-x})$. Outputs between -1 and 1. Zero-centered (unlike sigmoid which is 0 to 1), making it better for hidden layers than sigmoid. Still suffers from vanishing gradient for very large or small values. ReLU is generally preferred.

10 What is Linear activation?

Linear activation = $f(x) = x$. No transformation — output equals input. Used in the output neuron for regression problems where the prediction can be any real number (positive or negative). Using sigmoid/softmax here would wrongly restrict the output range.

[Training Concepts]

11 What is an Optimizer?

An optimizer is an algorithm that updates the model's weights to minimize the loss function. It uses the gradients computed during backpropagation. Common optimizers: Adam (adaptive learning rate — most popular), SGD (Stochastic Gradient Descent), RMSprop, Adagrad, Nadam, Adadelta.

12 What is Adam optimizer specifically?

Adam (Adaptive Moment Estimation) combines Momentum (tracks past gradient direction) and RMSprop (adapts learning rate per parameter). It maintains moving averages of gradients and squared gradients. Each weight gets its own adaptive learning rate. Converges faster and more reliably than plain SGD.

13 What are different Loss Functions?

MSE (Mean Squared Error): used for regression — averages squared differences, penalizes large errors. MAE (Mean Absolute Error): less sensitive to outliers. Binary Crossentropy: for 2-class classification. Categorical Crossentropy: for multi-class with one-hot labels. Sparse Categorical Crossentropy: for multi-class with integer labels. Hinge Loss: for SVM-style classifiers.

14 What is Backpropagation?

Backpropagation is the algorithm used to compute gradients of the loss with respect to every weight in the network. It applies the chain rule of calculus starting from the output layer and going backward to the input layer. These gradients tell the optimizer how much and in which direction to adjust each weight to reduce the loss.

15 What is Gradient Descent?

Gradient Descent is an iterative optimization algorithm. It updates weights in the direction opposite to the gradient (downhill): $w = w - \text{learning_rate} \times \text{gradient}$. Three variants: Batch GD (uses entire dataset — slow), SGD (one sample at a time — noisy), Mini-batch GD (uses small batches — balance of speed and stability — most commonly used).

16 What is Vanishing Gradient?

When gradients become extremely small as they are backpropagated through many layers, early layers stop learning because their weights barely change. Caused by sigmoid/tanh activation functions that squash values into small ranges. Fixed by using ReLU activation, batch normalization, and residual connections (skip connections in ResNet).

17 What is Exploding Gradient?

Opposite of vanishing — gradients grow exponentially large during backpropagation, causing weight updates to be massive and training to become unstable (loss becomes NaN). More common in RNNs. Fixed by gradient clipping (cap gradients at a maximum value) and proper weight initialization.

18 What is Dropout?

Dropout is a regularization technique where during training, a random fraction of neurons (e.g. 50% with Dropout(0.5)) are temporarily set to zero. This prevents neurons from co-adapting and forces the network to learn redundant representations. During inference/prediction, dropout is turned off — all neurons are active.

19 Other regularization techniques?

L1 Regularization (Lasso): adds sum of absolute weights to loss — encourages sparse weights (some become exactly 0). L2 Regularization (Ridge): adds sum of squared weights to loss — keeps weights small. Early Stopping: stop training when validation loss starts increasing. Data Augmentation: artificially expand training data. Batch Normalization: normalizes layer inputs.

20 What is Scaling and Normalization? Why necessary?

Scaling transforms feature values to a standard range. Min-Max Normalization: scales to [0,1] — formula: $(x - \min)/(\max - \min)$. Z-score Standardization (StandardScaler): mean=0, std=1 — formula: $(x - \text{mean})/\text{std}$. Necessary because features with large ranges (tax: 100-700) dominate gradient updates and make training slow, biased, and unstable.

21 What is the difference between ML approach and DNN approach?

ML (e.g. sklearn LinearRegression): simple, no hidden layers, only learns linear relationships, needs manual feature engineering, fast to train. DNN: multiple layers, learns non-linear and complex relationships automatically, no manual feature engineering needed, slower to train but much more powerful for complex data.

22 Why no feature engineering in DNN?

DNN hidden layers automatically learn feature interactions and non-linear transformations from raw data. The network essentially does its own feature engineering internally. Feature engineering is more critical for traditional ML models (linear regression, SVM) that cannot learn these transformations on their own.

ASSIGNMENT 2 — BINARY CLASSIFICATION USING DNN (IMDB SENTIMENT)

Dataset: IMDB Movie Reviews — Positive / Negative Classification

[Sequence Models — RNN, LSTM, GRU]

1 What is RNN (Recurrent Neural Network)?

RNN is a neural network designed for sequential data. Unlike a regular ANN, it has a hidden state (memory) that is passed from one timestep to the next. At each step, it takes the current input AND the previous hidden state to compute the output. This memory allows it to understand context and word order in text.

2 What is the problem with standard RNN?

Standard RNN suffers from the vanishing gradient problem for long sequences. When backpropagating through many timesteps, gradients become very small — the network forgets information from many steps ago. For a review of 500 words, the RNN may completely forget words from the beginning by the time it reaches the end.

3 What is LSTM (Long Short-Term Memory)?

LSTM solves the vanishing gradient problem using three gates: (1) Forget Gate — decides what past information to discard. (2) Input Gate — decides what new information to store. (3) Output Gate — decides what to output. LSTM also maintains a Cell State (long-term memory) separate from the hidden state. This allows it to remember information across hundreds of timesteps.

4 What is GRU (Gated Recurrent Unit)?

GRU is a simplified version of LSTM with only two gates: Reset Gate (how much past to forget) and Update Gate (how much past to carry forward). Fewer parameters than LSTM, trains faster, and performs similarly on most tasks. Preferred when data is limited or fast training is needed.

5 Types of RNN and their applications?

One-to-One: standard ANN (image classification). One-to-Many: one input → sequence output (image captioning — one image → caption sentence). Many-to-One: sequence → one output (sentiment analysis — review → positive/negative). Many-to-Many (same length): part-of-speech tagging. Many-to-Many (different): machine translation (English → French).

6 What is BRNN (Bidirectional RNN)?

Bidirectional RNN processes the sequence in both forward direction (left to right) and backward direction (right to left) simultaneously. The outputs from both directions are concatenated. This allows the model to see both past and future context for each word. More accurate than one-directional RNN for tasks like Named Entity Recognition.

7 Why use RNN/LSTM/GRU instead of regular ANN?

Text has sequential dependency — word order carries meaning. 'The movie was not good' and 'The movie was good' have opposite meanings. ANN treats each word independently and loses order. RNN/LSTM maintain memory of previous words, capturing this dependency. ANN also needs fixed-size input which is problematic for variable-length text.

8 What is an Embedding Layer?

An Embedding layer converts integer word indices into dense continuous vectors (word embeddings). Each word is represented as a vector of fixed size (e.g., 100 dimensions). Words with similar meanings get similar vectors — 'good' and 'great' are close in embedding space. Unlike one-hot encoding (sparse, 10,000 dims), embeddings are dense and semantically meaningful.

9 What is GlobalMaxPooling1D?

After the Embedding layer, each review is a 2D matrix (500 timesteps × 100 dimensions). GlobalMaxPooling1D takes the maximum value across all 500 timesteps for each of the 100 dimensions — output is a 1D vector of 100 values. It captures the most prominent features of the entire sequence regardless of position.

10 Why is padding done?

All reviews must have the same length for batch processing in neural networks. Since reviews have different word counts, shorter reviews are padded with zeros and longer ones are truncated to a fixed length (500 here). Padding is added at the beginning or end (pre/post). The network learns to ignore zero-padding.

11 Why is Sigmoid used in output for binary classification?

Sigmoid outputs a single value between 0 and 1, representing the probability of the positive class. If output > 0.5 → positive review; if < 0.5 → negative review. Binary crossentropy loss works with this probability. Softmax would be used if there were more than 2 classes.

12 Can ANN implement the same? Challenges?

Technically yes, but poorly. Challenges: (1) ANN treats words independently — loses sequential context and word order. (2) Needs fixed-size flattened input — destroys sequence structure. (3) Cannot handle variable-length sequences naturally. (4) Much lower accuracy on text tasks. LSTM/GRU are designed specifically for sequences.

13 What is Encoder-Decoder architecture?

Encoder-Decoder is used for sequence-to-sequence tasks. The Encoder reads the entire input sequence and compresses it into a fixed-size context vector (thought vector). The Decoder takes this vector and generates the output sequence step by step. Used in machine translation (English → French), text summarization, question answering, chatbots.

14 Recurrent Network vs Recursive Network?

Recurrent Network: processes data arranged in a linear sequence (text, time series) — same weights applied at each timestep from left to right. Recursive Network: processes data arranged in a tree/hierarchical structure (parse trees, nested sentences) — applies the same network function recursively at each node of the tree.

15 What is Binary Crossentropy loss?

Binary Crossentropy = $-[y \times \log(p) + (1-y) \times \log(1-p)]$. y = true label (0 or 1), p = predicted probability. When prediction is confident and correct, loss is near 0. When confident and wrong, loss is very high (log approaches infinity). This heavily penalizes wrong confident predictions, pushing the model to be accurate.

16 What is Dropout and why is it critical here?

In IMDB model, Dropout(0.5) is used after the Dense(256) layer. Without it, the model overfits — reaches 100% training accuracy but only 86.56% test accuracy. Dropout randomly deactivates 50% of neurons each training step, preventing memorization and forcing generalization. Disabled automatically during prediction/inference.

ASSIGNMENT 3 — CNN: FASHION MNIST CLASSIFIER

Dataset: Fashion MNIST — 10 clothing categories, 28×28 grayscale images

[CNN Core Concepts]

1 What is a CNN (Convolutional Neural Network)?

CNN is a deep learning architecture specifically designed for image data. It uses convolutional filters (kernels) that slide over the image to detect local patterns like edges, curves, and textures. Unlike ANN where every neuron connects to every neuron (fully connected), CNN neurons share weights through filters — drastically fewer parameters. CNNs are translation-invariant — they detect a feature wherever it appears in the image.

2 What is a Convolutional Layer?

A convolutional layer applies a set of learnable filters to the input. Each filter slides across the image performing element-wise multiplication and summing the results (dot product) — this is called convolution. Output is called a feature map. Multiple filters detect different features. Conv2D(32, 3×3) means 32 filters each of size 3×3.

3 How to calculate output image size after convolution?

Formula: $\text{Output Size} = (\text{Input Size} - \text{Filter Size} + 2 \times \text{Padding}) / \text{Stride} + 1$. Example without padding: Input=28×28, Filter=3×3, Padding=0, Stride=1 → Output = $(28 - 3 + 0) / 1 + 1 = 26$. So output is 26×26. With 32 filters → 26×26×32 feature maps. Each filter adds $3 \times 3 \times 1 + 1 = 10$ parameters.

4 What is a Filter / Kernel?

A filter (kernel) is a small matrix of learnable weights (e.g., 3×3 = 9 values). It slides over the input image and extracts a specific feature. One filter might detect horizontal edges, another vertical edges, another curves. These weights are learned during training via backpropagation — the network discovers what features are useful for the task automatically.

5 What is Stride?

Stride is the number of pixels the filter moves at each step. Stride=1 (default): filter moves 1 pixel at a time — more overlap, larger output, more detail. Stride=2: filter moves 2 pixels at a time — less overlap, smaller output, faster computation. Larger stride = smaller feature map = less spatial information retained.

6 What is Padding? Why is it required?

Padding adds extra rows/columns of zeros around the border of the input image. Two types: Valid Padding (no padding) — output is smaller than input (28→26). Same Padding — adds zeros to keep output same size as input. Required because: (1) without padding, edges are underrepresented — filter covers them less. (2) Network loses spatial dimensions rapidly in deep CNNs.

7 What are the advantages of padding?

1. Preserves spatial dimensions — output same size as input. 2. Border pixels are processed as many times as center pixels — edge features not lost. 3. Allows building deeper CNNs without shrinking feature maps to nothing. 4. Better feature extraction at image boundaries.

[Pooling]

8 What is a Pooling Layer?

A pooling layer reduces the spatial dimensions of feature maps. It slides a window (e.g., 2×2) over the feature map and summarizes each region with a single value. Pooling has no learnable parameters — it's a fixed operation. Placed after convolutional layers to progressively reduce size.

9 What is Max Pooling? What is Average Pooling?

Max Pooling: takes the maximum value in each window. Retains the strongest feature activation — most commonly used. Example: 2×2 max pool on a 26×26 map $\rightarrow 13 \times 13$. Average Pooling: takes the average of all values in each window. Smoother but may dilute strong activations. Global Average Pooling: averages the entire feature map into a single value.

10 Advantages and drawbacks of Pooling?

Advantages: (1) Reduces computation — smaller feature maps = fewer parameters in later layers. (2) Makes model translation-invariant — feature detected even if slightly shifted. (3) Reduces overfitting by providing abstraction. Drawbacks: (1) Loses precise spatial information (exact location of feature). (2) Max pooling discards non-maximum values which may contain useful info.

11 Types of Pooling?

Max Pooling: keeps maximum — best for detecting whether a feature is present anywhere in the region. Average Pooling: keeps average — smoother representation. Global Max Pooling: single max from entire feature map. Global Average Pooling (GAP): single average from entire feature map — used before output layer in modern CNNs. Min Pooling: rarely used.

[Dense / Flatten / Output]

12 What is the Flatten Layer? Why is it required?

Flatten converts a multi-dimensional tensor into a 1D vector. After Conv2D and MaxPooling, feature maps are 3D (e.g., $13 \times 13 \times 32 = 5408$ values). Dense (fully connected) layers require 1D input. Flatten reshapes $13 \times 13 \times 32 \rightarrow [5408]$. Without flatten, you cannot connect convolutional layers to fully connected layers.

13 What is a Fully Connected / Dense Layer?

A Dense layer connects every neuron in it to every neuron in the previous layer — fully connected. After flattening, Dense layers perform high-level reasoning on the extracted features. Dense(128, relu) = 128 neurons with ReLU. The final Dense layer gives class probabilities: Dense(10, softmax) for 10 clothing classes.

14 What are the 10 categories in Fashion MNIST?

0: T-shirt/top, 1: Trouser, 2: Pullover, 3: Dress, 4: Coat, 5: Sandal, 6: Shirt, 7: Sneaker, 8: Bag, 9: Ankle boot. Each class has 6,000 training and 1,000 test images. Images are 28×28 grayscale (pixel values 0-255, normalized to 0-1).

15 What activation functions are used and why?

ReLU in Conv2D and Dense hidden layers — avoids vanishing gradient, computationally efficient, enables deep learning. Softmax in final Dense(10) layer — converts 10 raw scores into probabilities summing to 1. The class with highest probability is the predicted clothing category.

16 Other popular MNIST datasets?

Original MNIST: 70,000 handwritten digit images (0-9), 28×28 grayscale — the 'hello world' of deep learning. EMNIST: extended MNIST with handwritten letters (A-Z) in addition to digits. KMNIST: Kuzushiji (old Japanese cursive characters). MNIST-C: corrupted/distorted versions for robustness testing. Fashion MNIST was created to replace original MNIST as a harder benchmark.

17 What is the drawback of using no padding (valid padding)?

Without padding: (1) Feature maps shrink after every convolution — with deep networks, maps become too small. (2) Border/edge pixels are covered by fewer filter positions than center pixels — edge features are underrepresented. (3) Information at image boundaries is progressively lost. That's why 'same' padding is used in many modern architectures.

ADDITIONAL TOPICS — GAN, DBN, RL

[Generative Adversarial Network (GAN)]

1 What is a GAN?

GAN (Generative Adversarial Network) is a framework of two neural networks trained simultaneously in competition. The Generator creates fake data trying to fool the Discriminator. The Discriminator tries to distinguish real data from fake. They compete until the Generator becomes so good that the Discriminator cannot tell real from fake — achieving Nash Equilibrium.

2 What are the blocks/components of GAN?

Generator (G): Takes random noise vector (z) as input and generates fake data (images, text, audio). Discriminator (D): Takes real or generated data as input and outputs a probability — how likely the input is real. Training: G tries to maximize D's error. D tries to minimize its own error. Loss: Generator loss = how often D correctly identifies fakes. Discriminator loss = binary crossentropy on real vs fake.

3 Applications of GAN?

Image generation (DeepFake faces — thispersondoesnotexist.com), Image-to-Image translation (horse→zebra, day→night — CycleGAN), Super Resolution (low-res → high-res images — SRGAN), Text-to-Image generation (DALL-E concept), Data Augmentation (generate training samples for rare classes), Drug discovery (generate molecular structures), Art generation.

[Deep Belief Network (DBN)]

4 What is a Deep Belief Network (DBN)?

A Deep Belief Network is a generative model made of multiple layers of Restricted Boltzmann Machines (RBMs) stacked on top of each other. Each RBM layer learns to represent features of the layer below it. Trained greedily — one layer at a time. DBNs can be used for feature extraction, dimensionality reduction, and as pre-training for deep networks. Largely replaced by CNNs and modern architectures today.

5 What is an RBM (Restricted Boltzmann Machine)?

RBM is a two-layer stochastic neural network: visible layer (input data) and hidden layer (learned features). 'Restricted' means no connections within the same layer — only between visible and hidden. It learns a probability distribution of the input data. Multiple RBMs stacked = Deep Belief Network. Used for collaborative filtering, feature learning, dimensionality reduction.

[Reinforcement Learning (RL)]

6 What is Reinforcement Learning?

Reinforcement Learning is a type of ML where an Agent learns to make decisions by interacting with an Environment. The agent takes Actions, receives Rewards (positive for good actions, negative for bad), and updates its policy to maximize cumulative reward over time. No labeled dataset — the agent learns from trial and error.

7 Key components of Reinforcement Learning?

Agent: the learner/decision-maker. Environment: the world the agent interacts with. State (S): current situation of the agent. Action (A): choices the agent can make. Reward (R): feedback signal after each action (positive=good, negative=bad). Policy: strategy mapping states to actions. Value Function: expected future reward from a state. Q-learning and Deep Q-Network (DQN) are popular RL algorithms.

8 Applications of Reinforcement Learning?

Game playing: AlphaGo (defeated world champion in Go), OpenAI Five (Dota 2). Robotics: teaching robots to walk, grasp objects. Autonomous vehicles: lane changing, path planning. Finance: algorithmic trading, portfolio optimization. Healthcare: personalized treatment plans. Recommendation systems: optimizing long-term user engagement.

[General DL Concepts — Quick Reference]

9 What is Epoch, Batch, Iteration?

Epoch: one complete pass through the entire training dataset. Batch: a subset of training data processed at once (batch_size=64 means 64 samples per update). Iteration: one weight update = one batch processed. If dataset=1000 samples, batch_size=64 → ~16 iterations per epoch. More epochs = more training = better learning (up to a point).

10 What is Overfitting and Underfitting?

Overfitting: model learns training data too well including noise — high train accuracy, low test accuracy. Caused by too complex model or too little data. Underfitting: model is too simple to capture patterns — low train and test accuracy. Perfect fit = good generalization. Solutions for overfitting: dropout, regularization, more data, early stopping.

11 What is Transfer Learning?

Transfer Learning reuses a pre-trained model (trained on a large dataset like ImageNet with millions of images) as the starting point for a new task. The pre-trained model's convolutional layers already know how to detect edges, textures, shapes. Only the final layers are retrained for the new task. Saves time and data — you don't need millions of images.

12 What is Batch Normalization?

Batch Normalization normalizes the output of each layer (makes mean=0, std=1) within each mini-batch during training. Benefits: (1) Allows higher learning rates — faster training. (2) Reduces sensitivity to weight initialization. (3) Acts as a regularizer — reduces overfitting. (4) Smooths the loss landscape. Applied before or after the activation function.

13 What is the difference between Classification and Regression?

Regression: predicts a continuous numerical value (house price = 24.5, temperature = 36.8°C). Output layer: 1 neuron, linear activation, MSE/MAE loss. Classification: predicts a class/category (cat or dog, positive or negative). Binary: sigmoid, binary crossentropy. Multi-class: softmax, categorical/sparse categorical crossentropy.

14 What is a Confusion Matrix?

A confusion matrix is a table showing model performance on a classification problem. For binary: True Positive (TP): correctly predicted positive. True Negative (TN): correctly predicted negative. False Positive (FP): predicted positive but actually negative (Type I error). False Negative (FN): predicted negative but actually positive (Type II error). Metrics: Precision = $TP/(TP+FP)$, Recall = $TP/(TP+FN)$, $F1 = 2 \times P \times R / (P+R)$.